Reviewer Pack — Minimal Symmetric Braid Length and Circuit Depth Inequality

Entry 0e98fc7c481f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 07:29:44 UTC

Minimal Symmetric Braid Length and Circuit Depth Inequality

Entry ID: `0e98fc7c481f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 07:29:44 UTC

1. Conjecture

Field A (mathematical branch): Braid Group Theory **Field B** (complexity object): Boolean Circuits (Circuit Depth)

Statement:

> For every boolean circuit C with n inputs, the minimal symmetric braid length of its associated link L_C is linearly correlated with its circuit depth d(C), such that msl(L_C) = Θ(d(C)).

Rationale (proposer's reasoning):

> The study of symmetric braids can provide a new perspective on understanding the complexity of boolean circuits. By linking this geometric structure to circuit complexity, it may reveal hidden structures or patterns in computational processes.

Taxonomy category: `braid_circuit_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `27a12b8ff0909f6f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all circuits C with n inputs (n ≤ 40), the correlation coefficient between minimal symmetric braid length msl(L_C) and circuit depth d(C) is ≥ 0.8, AND the mean of msl(L_C) across 30 seeds is within a factor of 3 of its median.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"symmetric braid length" AND "Boolean circuit depth"` - `"braid group theory" AND circuit complexity` - `"minimal symmetric braid length" ~ "circuit depth"`

Top relevant hits considered: - [http://arxiv.org/abs/2008.01820v2] Lower Bounds on Circuit Depth of the Quantum Approximate Optimization Algorithm - [http://arxiv.org/abs/2605.04748v1] Automated Circuit Depth Reduction of Quantum Subroutines via Compilation - [http://arxiv.org/abs/2306.00170v Reducing circuit depth with qubitwise diagonalization - [s2:10.1007/978-3-031-91107-1_6] Constructing Quantum Implementations with the Minimal T-depth or Minimal Width and Their Applications - [s2:10.1007/s10623-021-01005-z] Isomorphism of maximum length circuit codes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_circuit(n: int, depth: int):
        if n == 1:
            return ('input', 0)
        elif depth == 1:
            return ('or', [generate_random_boolean_circuit(1, 1), generate_random_boolean_circuit(
        else:
            gate = random.choice(['and', 'or'])
            inputs = [generate_random_boolean_circuit(n // 2, depth - 1) for _ in range(2)]
            return (gate, inputs)

    def calculate_minimal_symmetric_braid_length(circuit):
        if isinstance(circuit, tuple):
            gate, inputs = circuit
            if gate == 'input':
                return 1
            elif gate == 'and' or gate == 'or':
                return max(calculate_minimal_symmetric_braid_length(inp) for inp in inputs)
        else:
            raise ValueError("Invalid circuit format")

    def calculate_circuit_depth(circuit):
        if isinstance(circuit, tuple):
            gate, inputs = circuit
            if gate == 'input':
                return 1
```

```
            elif gate == 'and' or gate == 'or':
                return max(calculate_circuit_depth(inp) for inp in inputs) + 1
        else:
            raise ValueError("Invalid circuit format")

    n_max = 40
    instances_tested = 0
    msl_values = []
    depth_values = []

    for _ in range(30):
        n = random.randint(5, n_max)
        depth = random.randint(1, math.ceil(math.log2(n)))
        circuit = generate_random_boolean_circuit(n, depth)

        if isinstance(circuit, tuple):
            msl = calculate_minimal_symmetric_braid_length(circuit)
            d = calculate_circuit_depth(circuit)

            msl_values.append(msl)
            depth_values.append(d)
            instances_tested += 1

    correlation_coefficient = sum((msl - mean_msl) * (d - mean_depth) for msl, d in zip(msl_values
    mean_msl = sum(msl_values) / instances_tested
    median_msl = sorted(msl_values)[instances_tested // 2]

    if correlation_coefficient >= 0.8 and abs(mean_msl / median_msl - 1) <= 3:
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "correlation_coefficient=<{}> mean_msl_over_median=<{}>".format(correlati

    return {
        "metric_name": "minimal_symmetric_braid_length",
        "metric_value": mean_msl,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL: {}".format(result))
        results.append(result)

    mean_msl = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_msl) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_msl, std_dev, sup
    elif support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_msl, std_dev, sup
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print("RESULT: FALSIFIED counterexample=\"{}\" first_failing_seed={}".format(results[0]["c
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4c85c1b1.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4c85c1b1.py", line 69, in run_trial
    correlation_coefficient = sum((msl - mean_msl) * (d - mean_depth) for msl, d in zip(msl_values, dept
mean_msl) ** 2 for msl in msl_values)) * math.sqrt(sum((d - mean_depth) ** 2 for d in depth_values)))
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4c85c1b1.py", line 69, in <genexpr>
    correlation_coefficient = sum((msl - mean_msl) * (d - mean_depth) for msl, d in zip(msl_values, dept
mean_msl) ** 2 for msl in msl_values)) * math.sqrt(sum((d - mean_depth) ** 2 for d in depth_values)))
                                                             ^^^^^^^^
NameError: cannot access free variable 'mean_msl' where it is not associated with a value in enclosing
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and mean of minimal symmetric braid length against circuit depth. | next: Review the code for potential bugs that could cause a crash during execution and ensure it is capable of running to completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11824
2	preregistration	ollama_remote	glm4:latest	0	0	9726

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	8322
4	novelty	ollama_remote	glm4:latest	0	0	9876
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43815
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7987
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13855
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13589
9	judge	ollama_remote	glm4:latest	0	0	16835

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 135830 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0e98fc7c481f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e98fc7c481f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0e98fc7c481f.tar.gz` (if generated)